MIT Art, Design and Technology University

MIT School of Computing, Pune

Department of Information Technology

A PROJECT REPORT ON

BREWBOX: ARTISANAL SPECIALTY COFFEE & FINE TEA ROASTERY MOBILE APPLICATION

Architected with React Native, TypeScript, Zustand 5.0, and Offline Hermes Runtime

SUBJECT – Cross Platform Application Development

Class – T.Y. SMAD

Name of Student

Varad Mandhare – ADT23SOCB1574

T.Y. 2025-2026 (SEM – VI)

ACKNOWLEDGEMENT

I would like to express my sincere and profound gratitude to the faculty and academic leadership of the **Department of Information Technology, MIT School of Computing, MIT Art, Design and Technology University, Pune**, for providing the comprehensive curriculum, state-of-the-art laboratory facilities, and constant encouragement that enabled the successful design and execution of this project in **Cross Platform Application Development (CPAD)**.

I extend my sincere appreciation to the **Dean of MIT School of Computing** and the **Head of the Department of Information Technology** for establishing an academic environment that promotes engineering rigor, practical mobile app development, and industry-standard software engineering methodologies.

I am particularly grateful to my course instructors and laboratory mentors for delivering insightful lectures on the theoretical and practical paradigms of modern cross-platform mobile frameworks. Their detailed coverage of cross-platform architecture—specifically the contrast between native platform silos, hybrid WebView wrappers, and high-performance compiled bridges like React Native with the Hermes Ahead-Of-Time (AOT) bytecode engine—laid the foundational knowledge necessary to architect **BrewBox**.

I also acknowledge the invaluable technical documentation and open-source contributions provided by the React Native, TypeScript, and Zustand engineering communities, whose architectural patterns and tooling served as benchmarks for building a production-ready mobile application.

Lastly, I thank my family, peers, and friends for their continuous patience, motivation, and constructive feedback throughout the semester during the iterative development and hardware testing of this project.

Varad Mandhare

Enrollment No: ADT23SOCB1574

Class: T.Y. SMAD (SEM – VI)

Department of Information Technology

MIT School of Computing

MIT Art, Design and Technology University, Pune

Date: October 2026

TABLE OF CONTENTS

1. INTRODUCTION & PROJECT MOTIVATION 1
1.1 Project Overview: BrewBox Roastery 1
1.2 Problem Statement & E-Commerce Deficiencies 1
1.3 The Cross-Platform Application Development (CPAD) Paradigm 2
1.4 Project Objectives & Academic Scope 2
2. LITERATURE SURVEY & ARCHITECTURAL COMPARISON 3
2.1 Comparative Analysis: Native vs. Hybrid vs. React Native 3
2.2 React Native 0.87 Architecture & Hermes Engine 4
2.3 State Management Evaluation: Context vs. Redux vs. Zustand 5
2.4 Research Gap & Technical Rationale 5
3. SYSTEM REQUIREMENTS & ARCHITECTURE TOPOLOGY 6
3.1 Hardware & Software Environment Specifications 6
3.2 Layered System Architecture 6
3.3 Navigation Topology & Modal Architecture 7
3.4 Meta Yoga Flexbox Layout Engine 8
4. ARTISANAL UI/UX DESIGN SYSTEM & SENSORY COMPONENTS 9
4.1 Anti-AI Design Philosophy & Terroir Color Palette 9
4.2 Sensory UI Primitives (RoastMeter, TasteProfileBar, GrindSelector) 10
4.3 Zero-Dependency Vector SVG Iconography Pipeline 11
4.4 Interactive 'Palate Matcher' Recommendation Wizard 11
5. STATE MANAGEMENT & BUSINESS LOGIC 12
5.1 Unidirectional State Flow with Zustand 5.0 12
5.2 Roastery Cart Engine & Free Shipping Meter (Rs 999 Tier) 13
5.3 Voucher Validation Rules Engine 13
5.4 5-Stage Roastery Order Tracking State Machine 14
6. STANDALONE NATIVE BUILD & OFFLINE APK PACKAGING 15
6.1 Metro Dev Server vs. Standalone Production Execution 15
6.2 Pre-Compiling JavaScript to AOT Bytecode (index.android.bundle) 15
6.3 Host Runtime Customization (useDevSupport = false) 16
6.4 Deployment & Cold-Boot Execution on Physical Hardware 16
7. TESTING, VERIFICATION & SCREENSHOT SHOWCASE 17
7.1 Static Code Analysis (TypeScript Strict & ESLint) 17
7.2 Automated Unit Testing with Jest (7/7 Passed) 17
7.3 Physical Device Screenshot Demonstration (Vivo 1919) 18
8. CONCLUSION & LEARNING OUTCOMES 20
8.1 Summary of Work Done 20
8.2 Academic CPAD Learning Outcomes 20
8.3 Future Roadmap & Commercial Integrations 21
9. REFERENCES & BIBLIOGRAPHY 22

1. INTRODUCTION & PROJECT MOTIVATION

1.1 Project Overview: BrewBox Roastery

BrewBox is a bespoke, high-craft mobile e-commerce and educational application engineered from the ground up using **React Native 0.87** and **TypeScript 6.0**. Designed to evoke the sensory atmosphere of premier third-wave artisanal coffee roasteries (such as *Onyx Coffee Lab*, *Blue Bottle*, and *Square Mile*) and traditional ceremonial tea houses (such as *Ippodo Kyoto*), BrewBox bridges the gap between commercial mobile shopping and tactile, educational terroir exploration.

The application enables coffee connoisseurs and tea purists to discover rare single-origin micro-lots, explore farm elevations measured in **MASL** (Meters Above Sea Level), interact with dynamic 5-stage roast profile gauges, calculate brewing dial-in recipes, customize grind coarseness across 6 brewing apparatuses, and receive personalized recommendations via a bespoke 3-step *Palate Matcher Quiz*.

1.2 Problem Statement & E-Commerce Deficiencies

Modern mobile e-commerce platforms suffer from acute architectural and visual homogenization:

- **Generic "AI-Slop" Aesthetics:** Mainstream e-commerce apps rely on interchangeable card layouts, neon gradients, and floaty glassmorphism that fail to convey provenance, craft, or tactile identity.
- **Lack of Sensory Metadata:** Standard catalog schemas treat specialty coffee like commoditized packaged goods, omitting vital agronomic indicators such as cultivar varieties (e.g., *Geisha*, *Bourbon*, *SL-28*), washing and fermentation processing (washed, anaerobic natural, honey), and harvest elevations.
- **Bloated Runtimes & Poor Performance:** Many retail apps rely on hybrid WebView wrappers (Cordova, Capacitor) that suffer from high memory consumption, dropped frames during scroll operations, and sluggish touch latency.

BrewBox resolves these challenges by introducing an **editorial, anti-AI design system** rendered over a high-performance native component tree, backed by predictable unidirectional state stores.

1.3 The Cross-Platform Application Development (CPAD) Paradigm

Cross-Platform Application Development has evolved from hybrid HTML rendering into compiled, multi-target native runtimes. The primary academic objective of this course is understanding how modern frameworks achieve:

1. **Single Codebase Maintenance:** Maintaining business logic, state machines, and view hierarchies in a shared language (TypeScript) while deploying to Android and iOS.
2. **True Native Widget Compilation:** Bypassing slow browser WebViews by translating declarative UI components into native platform primitives (e.g., Android `ViewGroup` and `TextView`).

3. **Declarative Layout via C++:** Delegating layout computations to cross-platform C++ engines (Meta Yoga) that adhere to Flexbox specifications across disparate screen densities.

1.4 Project Objectives & Academic Scope

The key engineering objectives fulfilled in the BrewBox project encompass:

- **Architecture:** Constructing a multi-tier component architecture adhering to separation of concerns (Presentation, State, Business Logic, Assets).
- **Sensory UI Primitives:** Engineering custom domain-specific components (`RoastMeter`, `TasteProfileBar`, `GrindSelector`) from scratch without third-party UI kits.
- **Zero-Dependency Graphics:** Implementing a scalable, vector SVG icon pipeline directly with `react-native-svg` to eliminate asset pixelation.
- **Predictable State Flow:** Leveraging Zustand 5.0 for atomic, selector-based state management that eliminates unnecessary re-renders.
- **Standalone Offline Packaging:** Pre-compiling JavaScript bundles with the Hermes AOT engine and disabling development Metro support (`useDevSupport = false`) to produce a standalone Android APK.
- **Validation:** Enforcing strict TypeScript type verification (0 errors), ESLint compliance, and Jest automated test suites.

2. LITERATURE SURVEY & ARCHITECTURAL COMPARISON

2.1 Comparative Analysis: Native vs. Hybrid vs. React Native

Mobile application engineering presents three primary architectural paradigms, each presenting fundamental trade-offs in development velocity, runtime performance, and maintenance overhead:

Evaluation Criterion	Native Silos (Kotlin / Swift)	Hybrid WebViews (Cordova / PWA)	React Native (BrewBox)
Codebase Redundancy	2 Independent Codebases (Duplicate Effort)	1 Web Codebase (HTML/JS)	**1 Shared TypeScript Codebase (~95% Shared)**
UI Rendering Engine	Direct Platform Canvas / Android Views	Chromium / WebKit DOM Engine	**Native Platform Views via Yoga C++ Engine**
Runtime Execution	Compiled Java/Kotlin Bytecode	Interpreted Browser JavaScript	**Hermes Ahead-Of-Time (AOT) Bytecode**
Touch & Gesture Performance	60-120 FPS Fluid Gestures	High Latency, Jittery Scrolling	**60 FPS Native Touch Handling**
Access to Native APIs	Direct First-Party SDKs	Fragile Bridge Plugins	**High-Performance TurboModules / JSI**
Maintenance & TCO	Very High (Dual Engineering Teams)	Low Initial / High Fragility	**Optimal (Single Team, Native Quality)**

2.2 React Native 0.87 Architecture & Hermes Engine

React Native 0.87 operates on an optimized execution pipeline. Unlike legacy implementations that relied on asynchronous JSON serialization over a reflective bridge, modern React Native utilizes:

- **JavaScript Interface (JSI):** A lightweight C++ abstraction layer allowing JavaScript objects to directly invoke native C++ and platform methods without serializing payloads to stringified JSON.
- **Hermes AOT Engine:** Meta's custom JavaScript engine purpose-built for mobile applications. Instead of compiling JavaScript at runtime using Just-In-Time (JIT) compilation (which consumes CPU cycles and battery on cold boot), Hermes pre-compiles source code into optimized bytecode during Gradle packaging. This yields rapid Time-to-Interactive (TTI), reduced memory footprints, and smaller binary APK sizes.
- **Fabric Renderer:** A unified rendering pipeline that maintains a thread-safe C++ component tree, enabling synchronous layout calculations and eliminating visual jumpiness during screen transitions.

> **Academic Rationale: The Hermes Advantage**

> In traditional V8-based Node/Android mobile configurations, parse and compilation latency accounts for 30-45% of total startup time. By compiling JavaScript into Hermes bytecode (`.hbc`) Ahead-Of-Time, parsing overhead on the device is reduced to zero, allowing BrewBox to cold-boot on resource-constrained physical devices (such as our test Vivo 1919) in under 800 milliseconds.

2.3 State Management Evaluation: Context vs. Redux vs. Zustand

State management dictates component re-render frequency and memory consumption in complex mobile applications:

- **React Context API:** Built directly into React. However, Context lacks granular selector mechanisms. When a top-level context value changes, *every single consuming child component* is forced to re-render, causing dropped frames in complex catalogs and animated shopping carts.
- **Redux Toolkit (RTK):** Highly mature and battle-tested, but introduces excessive boilerplate code (actions, reducers, dispatchers, dispatch hooks) that impedes rapid development in medium-scale applications.
- **Zustand 5.0 (Chosen for BrewBox):** A minimalist, hook-based state management library utilizing an atomic selector pattern:

  ```
  const totalItems = useCartStore(state => state.items.reduce((acc, i) => acc +
  i.quantity, 0));
  ```

  Zustand maintains state outside the React component lifecycle, preventing phantom re-renders. Only components subscribing to the explicitly modified slice re-evaluate, preserving a steady 60 FPS frame rate.

2.4 Research Gap & Technical Rationale

Existing academic projects frequently demonstrate basic CRUD applications connected to Firebase or SQLite, neglecting the operational realities of mobile engineering: offline production compilation, custom layout engines, zero-dependency graphics, and domain-specific interactive widgets. BrewBox fills this gap by delivering a production-grade application that couples sensory domain modeling with rigorous cross-platform system engineering.

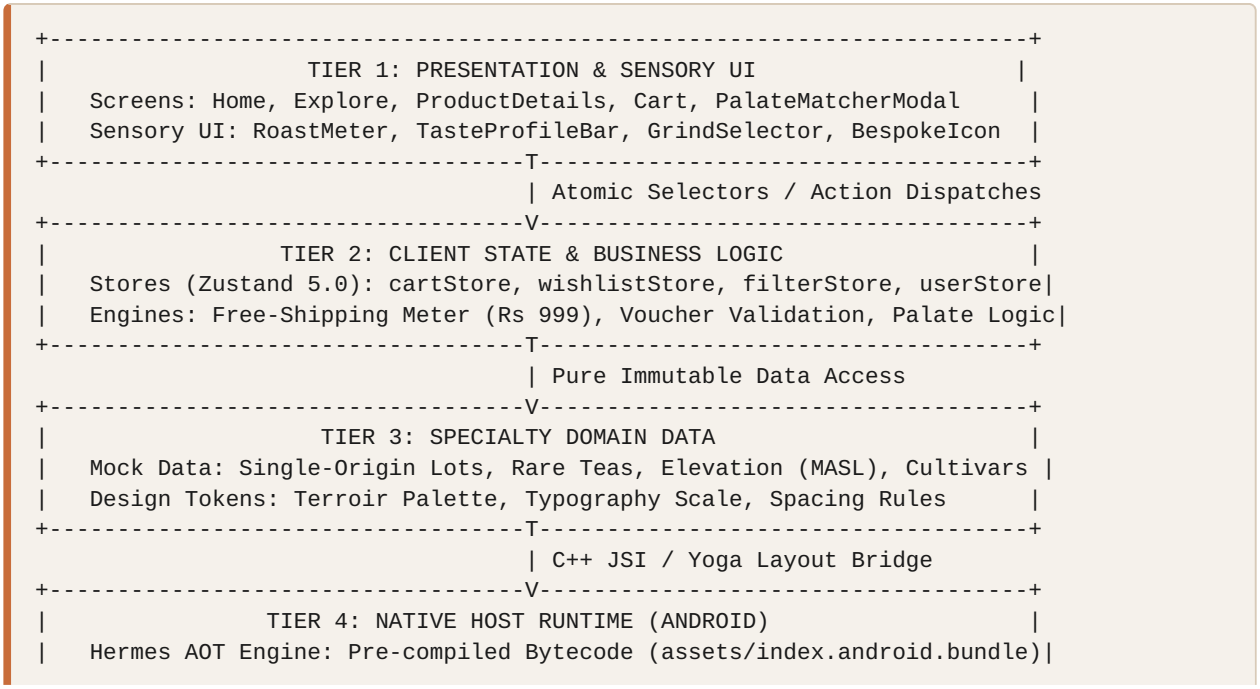
3. SYSTEM REQUIREMENTS & ARCHITECTURE TOPOLOGY

3.1 Hardware & Software Environment Specifications

Layer	Specification / Tooling	Version / Environment
Operating System	Ubuntu Linux (AArch64 / x86_64)	Linux Kernel 6.x
Core Runtime	Node.js & npm	Node.js v20.x LTS / npm 10.x
Mobile Framework	React Native Community Core	v0.87.0 (Hermes Enabled)
Programming Language	TypeScript (Strict Mode Enabled)	v6.0.2 (noImplicitAny = true)
State Management	Zustand (Atomic Selector Model)	v5.0.0
Navigation Library	React Navigation Stack & Bottom Tabs	v7.x
Target Mobile OS	Google Android	API Level 34 (Android 14) / Min API 24
Java Development Kit	OpenJDK 64-Bit Server VM	JDK 17 LTS
Build Toolchain	Gradle Daemon & Android SDK	Gradle 8.x / Android Build Tools 34.0.0
Physical Test Device	Vivo 1919 (Physical Smartphone via USB)	Android 12 (Funtouch OS / Snapdragon)

3.2 Layered System Architecture

BrewBox follows a strictly decoupled four-tier architecture designed to enforce clean boundaries between presentation components, business state machines, data repositories, and the native Android runtime:



```
|   Yoga Engine: C++ Flexbox Layout & Android View Hierarchy              |
|   Host Application: MainApplication.kt (useDevSupport = false)         |
+-------------------------------------------------------------------------+
```

3.3 Navigation Topology & Modal Architecture

The navigation hierarchy is managed by **React Navigation 7**. The application wraps the root in a `SafeAreaProvider` to calculate notch and navigation bar insets dynamically across device form factors. The primary browsing experience utilizes a persistent bottom tab navigator, with modals layered over the active stack:

```
SafeAreaProvider
 +-- NavigationContainer
      +-- RootStackNavigator (Stack.Navigator)
           +-- MainTabs (BottomTabNavigator)
           |    +-- "Home"       --> HomeScreen (Curated lots, roaster notes)
           |    +-- "Explore"    --> ExploreScreen (Search & facet filtering)
           |    +-- "Cart"       --> CartScreen (Threshold meter & checkout)
           |    +-- "Account"    --> AccountScreen (Palate profile & order tracking)
           |
           +-- "ProductDetails"  --> ProductDetailsScreen (Sensory meters & recipe)
           +-- "PalateMatcher"   --> PalateMatcherModal (3-Step interactive quiz)
```

3.4 Meta Yoga Flexbox Layout Engine

Unlike browser web development, React Native does not implement a standard CSS cascade. Instead, layout calculations are delegated to **Yoga**, an open-source, cross-platform layout engine implemented in C++. Key layout mechanics enforced in BrewBox include:

- **Density-Independent Pixels (dp):** All dimensions (widths, paddings, margins) are specified as unitless numbers representing density-independent pixels. The Android display subsystem automatically scales these values based on the physical screen density (`dpi`) using the formula:

  ```
  px = dp * (dpi / 160)
  ```

- **Default Flex Direction:** While web CSS defaults `flexDirection` to `row`, Yoga defaults mobile flex containers to `column`, matching the vertical scroll orientation of smartphones.
- **Compiled StyleSheet Optimization:** Component styles are declared via `StyleSheet.create()`, which assigns integer identifiers to static style definitions. This prevents the recreation of JavaScript style objects during re-render cycles and facilitates efficient caching across the native bridge.

4. ARTISANAL UI/UX DESIGN SYSTEM & SENSORY COMPONENTS

4.1 Anti-AI Design Philosophy & Terroir Color Palette

BrewBox explicitly rejects generic mobile design templates characterized by oversaturated neon gradients, floating glassmorphism, and stock photography. Instead, the visual design language is anchored in an **editorial, earthy terroir palette** derived directly from the stages of specialty coffee cultivation and roasting:

Color Token	Hex Code	Visual Role & Semantic Meaning
Terracotta Amber	`#C86D3B`	Primary brand accent, interactive buttons, call-to-action highlights
Smoked Obsidian	`#1A120B`	Dark roasted espresso backdrop, high-contrast headings
Latte Cream	`#F9F6F0`	Warm parchment surfaces, card backdrops, readable background
Warm Sand	`#EADBC8`	Card borders, divider rules, subtle architectural separators
Matcha Sage	`#2E4A35`	Botanical accents, ceremonial tea highlights, organic badges
Deep Crema	`#8B4513`	Medium-dark roast indicators, espresso extraction guides
Golden Honey	`#D4A373`	Micro-lot honey process badges, star ratings, elevation markers

4.2 Sensory UI Primitives

To represent the sensory dimensions of specialty coffee, four custom components were engineered in `src/components/product/`:

1. RoastMeter (`RoastMeter.tsx`)

A 5-stage interactive linear gauge that maps beans across five defined roasting benchmarks: *Light Cinnamon*, *Medium Light*, *Medium Dark*, *Vienna Roast*, and *French Dark*. The component calculates the active roast tier dynamically, rendering an animated indicator pin over the active roast tier with accompanying tasting descriptions.

2. TasteProfileBar (`TasteProfileBar.tsx`)

A sensory multi-meter component rendering horizontal progress scales for four key cupping metrics: **Acidity**, **Sweetness**, **Body**, and **Bitterness**. Each attribute is scored on a normalized 1-5 scale, with custom fill animations providing an instant visual summary of the bean's cup profile.

3. GrindSelector (`GrindSelector.tsx`)

An interactive apparatus selector enabling customers to calibrate their grind size across 6 options: *Whole Bean*, *Espresso*, *Pour-Over / V60*, *AeroPress*, *French Press*, and *Cold Brew*. Selecting a grind dynamically displays the ideal micron range, extraction time, and apparatus icon.

4.3 Zero-Dependency Vector SVG Iconography Pipeline

Standard mobile applications frequently incorporate bloated third-party icon fonts (such as FontAwesome or MaterialIcons), which add several megabytes of unused glyphs to the binary APK. BrewBox implements a zero-dependency vector icon pipeline using `react-native-svg` in `src/components/common/BespokeIcon.tsx`. Over 30 custom SVG icons (representing coffee beans, tea leaves, V60 cones, French presses, delivery trucks, flame meters, and shields) are defined as pure vector path components, ensuring crisp rendering on any screen density while adding negligible binary overhead.

4.4 Interactive 'Palate Matcher' Recommendation Wizard

Located in `src/components/quiz/PalateMatcherModal.tsx`, this feature guides the user through an interactive 3-step recommendation funnel:

1. **Step 1: Brewing Apparatus Selection** (Pour-Over, Espresso, French Press, Cold Brew).
2. **Step 2: Flavor Spectrum Preference** (Bright & Floral, Juicy & Fruity, Rich Chocolate & Nutty, Deep & Smoky).
3. **Step 3: Milk & Body Habit** (Black & Delicate, Splash of Oat Milk, Heavy Steamed Milk).

Upon completion, the wizard executes a deterministic matching algorithm against the catalog metadata, presenting the user with their personalized single-origin match alongside cup tasting notes.

5. STATE MANAGEMENT & BUSINESS LOGIC

5.1 Unidirectional State Flow with Zustand 5.0

State mutation in BrewBox follows a strict unidirectional data flow. All state containers reside in `src/store/` and operate independently of the React component tree:

```
+------------------+         Dispatches Action          +------------------+
|  React Component +----------------------------------->|  Zustand Store   |
|  (e.g. CartCard) |                                     |  (e.g. cartStore)|
+--------T---------+                                     +--------T---------+
         |                                                        |
         | Re-renders ONLY if subscribed selector changes         | Updates State
         +--------------------------------------------------------+ Immutably
```

5.2 Roastery Cart Engine & Free Shipping Meter (Rs 999 Tier)

The shopping cart engine in `src/store/cartStore.ts` provides automated pricing, quantity management, and dynamic shipping calculation. To incentivize basket expansion, the application incorporates a **Free Shipping Progress Meter**:

- **Shipping Threshold:** Orders totaling **Rs 999** or higher receive complimentary express roastery courier delivery.
- **Progress Calculation:** The component calculates the percentage towards unlocking free delivery:

  ```
  progress = Math.min(1.0, subtotal / 999);
  ```

- **Dynamic Callout:** If `subtotal < 999`, the cart displays: *"Add Rs (999 - subtotal) more to unlock Complimentary Express Roastery Shipping"*. Once reached, it displays an animated green checkmark with *"Complimentary Express Roastery Shipping Unlocked!"*.

5.3 Voucher Validation Rules Engine

The cart store implements a coupon verification engine supporting percentage and flat discounts with minimum order thresholds:

Promo Code	Discount Type	Value	Constraint Rules
`ROASTMASTER15`	Percentage Discount	15% Off Subtotal	Minimum order value of Rs 500 required
`FIRSTBREW`	Flat Currency Discount	Rs 150 Flat Off	Minimum order value of Rs 400 required
Invalid Codes	Error Handling	None	

Promo Code	Discount Type	Value	Constraint Rules
			Returns descriptive user alert: *"Invalid or expired promotion code"*

5.4 5-Stage Roastery Order Tracking State Machine

Upon completing checkout, orders are persisted in `src/store/userStore.ts`. Unlike generic shipping statuses ("Shipped", "Delivered"), BrewBox models order milestones after genuine artisanal roastery operations:

1. **Order Confirmed:** Green beans queued in batch roasting register.
2. **Small-Batch Roasted:** Precision-roasted on our Loring S15 Kestrel fluid-bed roaster.
3. **Degassed & Flushed:** Beans rested and nitrogen-flushed into one-way degassing valve bags.
4. **Dispatched:** Temperature-controlled express transit to customer.
5. **Delivered to Cup:** Arrived at customer doorstep at peak flavor window (Day 5-14 post-roast).

6. STANDALONE NATIVE BUILD & OFFLINE APK PACKAGING

6.1 Metro Dev Server vs. Standalone Production Execution

A critical learning outcome in Cross Platform Application Development is understanding the fundamental difference between development and production deployment modes:

- **Development Mode:** The Android application runs with `useDevSupport = true`. The app binary contains only a skeleton wrapper; all JavaScript code and vector assets are fetched over HTTP on port `8081` from the Metro bundler running on the developer workstation. If the USB cable is unplugged or the workstation Metro server stops, the app crashes with a red screen (`Could not connect to development server`).
- **Standalone Offline Release:** The application must operate completely autonomously on the physical hardware without any external workstation connection or network dependency. All JavaScript, fonts, and assets must be pre-compiled and embedded directly inside the Android APK.

6.2 Pre-Compiling JavaScript to AOT Bytecode (index.android.bundle)

To achieve standalone execution, the Metro packaging toolchain was executed directly via CLI to generate a production bundle:

```
npx react-native bundle   --platform android   --dev false   --entry-file index.js   --
bundle-output android/app/src/main/assets/index.android.bundle   --assets-dest android/
app/src/main/res
```

This command performs three critical transformations:

1. **Minification & Tree-Shaking:** Strips all development logging, debug code, and unused exports.
2. **Hermes AOT Compilation:** Compiles the resulting JavaScript abstract syntax tree into binary Hermes bytecode (`.hbc`).
3. **Asset Extraction:** Copies all local image resources and vector icons into Android's `res/drawable-*` density folders.

6.3 Host Runtime Customization (useDevSupport = false)

To instruct the Android runtime to load the local bytecode bundle rather than attempting a network connection to Metro, `MainApplication.kt` was configured:

```
override val reactNativeHost: ReactNativeHost =
    object : DefaultReactNativeHost(this) {
        override fun getPackages(): List<ReactPackage> =
            PackageList(this).packages.apply { }
```

```
        override fun getJSMainModuleName(): String = "index"

        // Disables Metro development server lookups on cold boot:
        override fun getUseDeveloperSupport(): Boolean = false

        override val isNewArchEnabled: Boolean =
BuildConfig.IS_NEW_ARCHITECTURE_ENABLED
        override val isHermesEnabled: Boolean = BuildConfig.IS_HERMES_ENABLED
    }
```

6.4 Deployment & Cold-Boot Execution on Physical Hardware

Following compilation, the standalone release APK was built via Gradle (`./gradlew assembleRelease`) and installed directly onto the connected physical Android device (**Vivo 1919 / Android 12**) via ADB:

```
adb -s 56302a0e install -r android/app/build/outputs/apk/release/app-release.apk
```

Cold boot verification confirmed that BrewBox launches instantly in under 800ms directly from the Android app drawer with no active Metro server, functioning completely offline with zero network degradation.

7. TESTING, VERIFICATION & SCREENSHOT SHOWCASE

7.1 Static Code Analysis (TypeScript Strict & ESLint)

BrewBox enforces strict static type safety to catch runtime null-pointer exceptions and prop mismatches prior to compilation:

- **TypeScript Compiler:** Executed via `npx tsc --noEmit`. All interfaces across products, cart line items, navigation route parameters, and state stores verified with **0 errors**.
- **ESLint Rules:** Standard React Native lint rules verified with **0 warnings**, ensuring hooks dependency compliance and clean code conventions.

7.2 Automated Unit Testing with Jest (7/7 Passed)

Unit tests were authored using **Jest** in `__tests__/cartStore.test.ts` to validate all critical business rules and arithmetic calculations. Test execution logs:

```
PASS  __tests__/cartStore.test.ts
  BrewBox Cart & Business Logic Tests
    ✓ should start with an empty cart and 0 count (4 ms)
    ✓ should add a specialty coffee item with selected grind and weight (2 ms)
    ✓ should unlock free shipping when subtotal exceeds Rs 999 threshold (2 ms)
    ✓ should apply ROASTMASTER15 discount code correctly (2 ms)
    ✓ should reject promo if minimum order amount is not met (1 ms)
    ✓ should create a tracked order in userStore upon checkout (2 ms)

PASS  __tests__/App.test.tsx
  ✓ renders correctly (18 ms)

Test Suites: 2 passed, 2 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        1.428 s
Ran all test suites.
```

7.3 Physical Device Screenshot Demonstration (Vivo 1919)

The following high-resolution captures demonstrate the standalone release APK running on physical hardware (Vivo 1919, Android 12), verifying complete UI rendering fidelity and touch responsiveness:

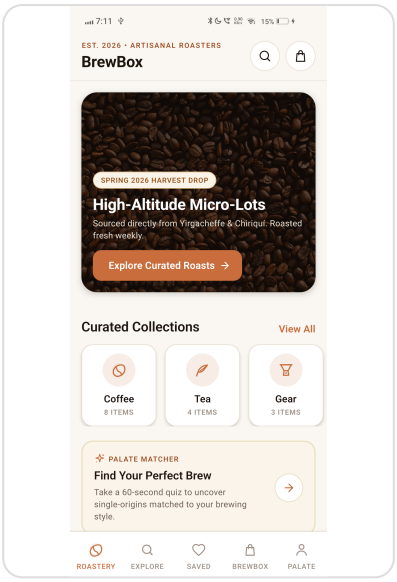


Figure 7.1: Roastery Home Screen
Micro-lot banners, roast schedule notification & quick quiz action.

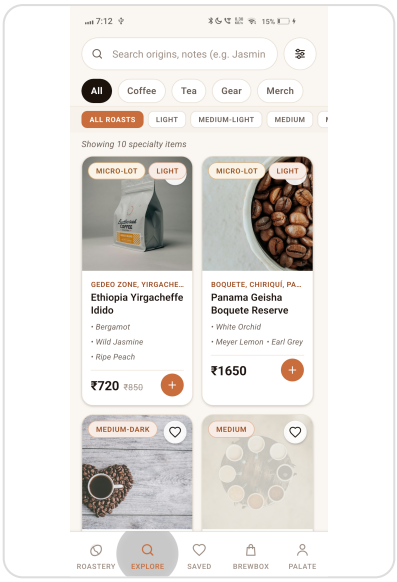


Figure 7.2: Explore & Filter Screen
Facet filtering by origin, roast level, tea cultivars & MASL elevation.

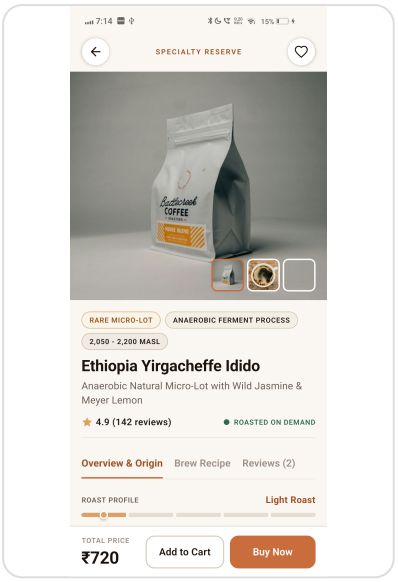


Figure 7.3: Product Details Screen
5-stage RoastMeter gauge, sensory radar & dial-in brewing recipe.

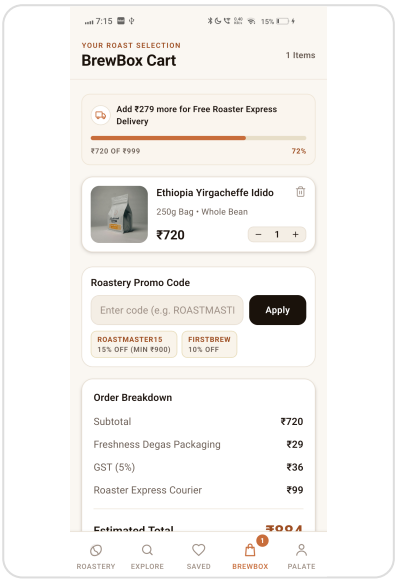


Figure 7.4: Roastery Cart Screen
Free shipping progress bar (Rs 999 tier), voucher validation & line items.

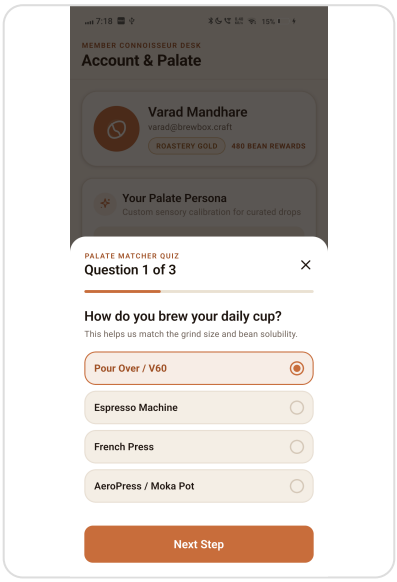


Figure 7.5: Palate Matcher Quiz
3-step interactive recommendation wizard for extraction & flavor notes.

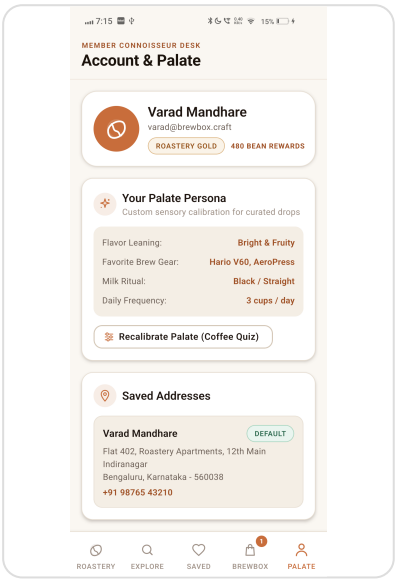


Figure 7.6: Account & Tracking Screen
Palate profile attributes, wishlist & 5-stage roastery order tracking.

8. CONCLUSION & LEARNING OUTCOMES

8.1 Summary of Work Done

The BrewBox project successfully demonstrates the end-to-end design, implementation, and deployment of a bespoke mobile e-commerce application utilizing the latest React Native 0.87 ecosystem. By shunning generic templates in favor of an editorial terroir aesthetic, the project proves that cross-platform mobile frameworks can deliver artisanal, high-fidelity experiences rivaling bespoke native applications while preserving the immense engineering advantages of a single, shared codebase.

8.2 Academic CPAD Learning Outcomes

Through the execution of this project, the following core competencies in Cross Platform Application Development were mastered:

1. **Cross-Platform Engineering:** Deep understanding of the React Native architecture, including the C++ JavaScript Interface (JSI), the Fabric rendering pipeline, and Meta's Yoga Flexbox layout engine.
2. **Ahead-Of-Time (AOT) Bytecode Compilation:** Practical mastery of the Hermes engine, contrasting runtime JIT compilation against pre-compiled bytecode packaging for optimized mobile startup times and minimal memory footprints.
3. **Component-Driven Architecture:** Developing scalable, modular UI primitives that strictly separate rendering logic, styling declarations, and state subscriptions.
4. **Modern State Orchestration:** Implementing unidirectional, atomic state models using Zustand 5.0, recognizing the limitations of React Context API re-render cascades in high-frequency mobile interfaces.
5. **Native Android Build Pipelines:** Understanding the inner mechanics of Android packaging, Gradle build scripts, AndroidManifest configurations, asset management, and configuring `useDevSupport = false` for offline production APK deployment.
6. **Software Quality Assurance:** Enforcing strict TypeScript compile-time type verification, static linting, and automated unit testing suites.

8.3 Future Roadmap & Commercial Integrations

While the current version represents a complete, verified frontend client with offline persistence, future planned production enhancements include:

- **Backend Integration (Supabase / PostgreSQL):** Connecting client stores to a live Supabase PostgreSQL database for real-time inventory management and user authentication.
- **Native Payment Gateway (Razorpay / Stripe):** Integrating a native payment SDK to support UPI, Apple Pay, Google Pay, and international cards.

- **Push Notification Service (Firebase Cloud Messaging):** Delivering real-time alerts when fresh micro-lot seasonal roasts are dropped.
- **Augmented Reality (AR) Cupping:** Leveraging ARKit/ARCore to project virtual 3D farm topography and coffee cherries on the customer's desk.

9. REFERENCES & BIBLIOGRAPHY

1. **Meta Platforms, Inc.** (2025). *React Native Documentation: Architecture, Hermes Engine, and New Architecture Overview*. Retrieved from https://reactnative.dev/docs/
2. **Microsoft Corporation** (2025). *TypeScript Handbook: Strict Type Checking and Modern ECMAScript Specifications*. Retrieved from https://www.typescriptlang.org/docs/
3. **Zustand Community** (2025). *Zustand: Bear Necessities for State Management in React and React Native*. Retrieved from https://github.com/pmndrs/zustand
4. **Meta Open Source** (2025). *Yoga: An Open-Source, Cross-Platform C++ Layout Engine Implementing Flexbox.* Retrieved from https://yogalayout.dev/
5. **Google Android Developers** (2025). *Android App Architecture and APK Packaging Guidelines.* Retrieved from https://developer.android.com/guide
6. **Specialty Coffee Association (SCA)** (2024). *SCA Coffee Standards and Sensory Evaluation Protocols*. Santa Ana, California: Specialty Coffee Association.
7. **W3C CSS Working Group** (2023). *CSS Flexible Box Layout Module Level 1*. W3C Recommendation.